

LET'S
ROCK

THE
FUTURE

FIAP

GRADUAÇÃO

CURSOS PRESENCIAIS E ON-LINE

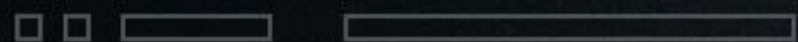
Prof. Dr. Marcus Grilo

Prof. Dr. Marcus Grilo

Ciência da Computação

Computer Organization and Architecture

Representação de imagens: binário, hexadecimal
e saída digital



Prof. Dr. Marcus Grilo



<https://www.linkedin.com/in/marcus-grilo/>



profmarcus.grilo@fiap.com.br

Objetivo

Ao final da aula, você deverá ser capaz de:

Uma imagem digital é formada por **pixels**;

Cada pixel pode ser representado por números;

Números podem ser armazenados em **binário**;

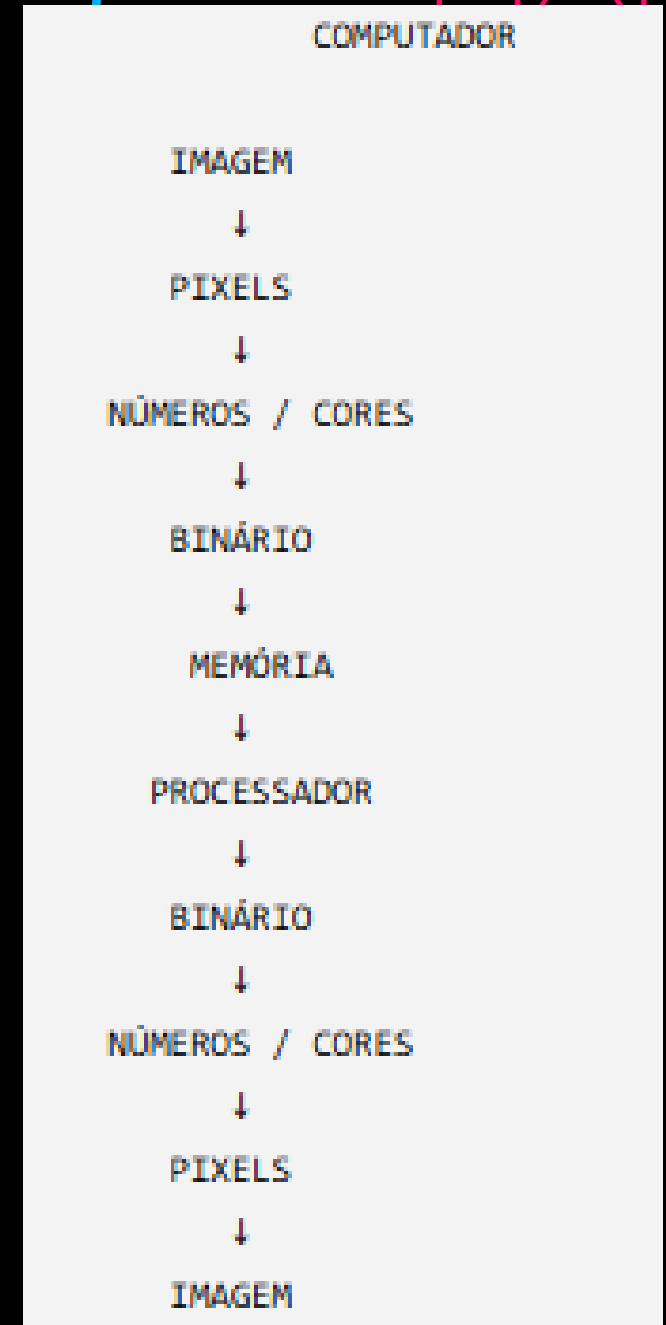
Uma sequência de bits pode representar uma imagem;

O processador lê esses dados da memória e pode reconstruir a imagem;

Uma matriz de LEDs pode funcionar como uma representação física de uma imagem

1 - "Como o computador sabe que um conjunto de 0 e 1 representa uma imagem?"

Porque existe uma convenção que determina como aqueles bits devem ser interpretados.



2 - Começando com uma imagem extremamente simples

FIAP

Para não complicar inicialmente com RGB, JPEG etc., vamos utilizar uma imagem de **8×8 pixels em preto e branco.**

Cada pixel terá apenas duas possibilidades:

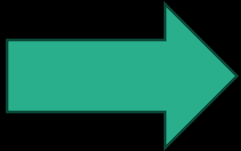
0 = preto

1 = branco

2 - Começando com uma imagem extremamente simples FIAP

Por exemplo, podemos representar a letra **A**:

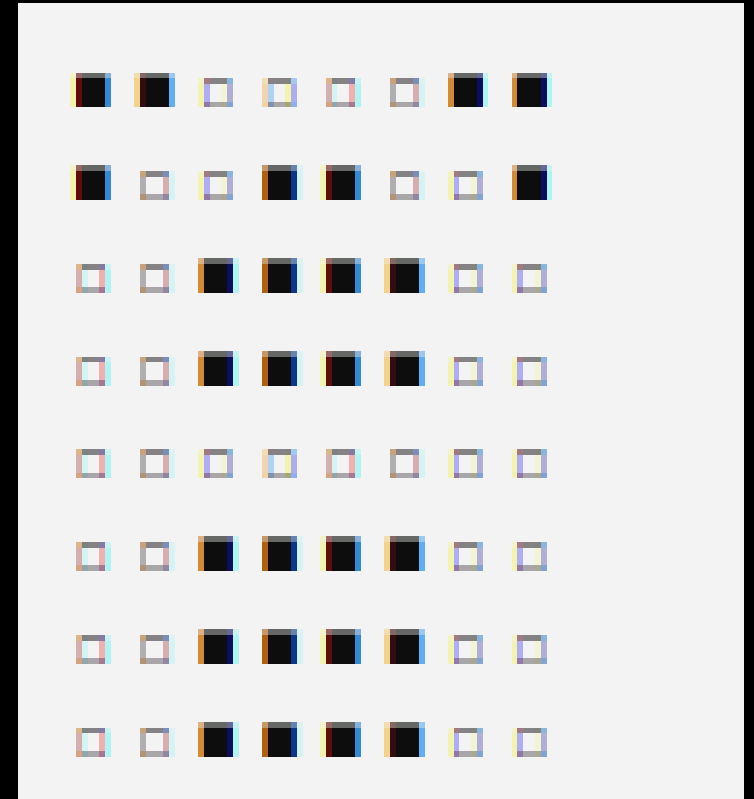
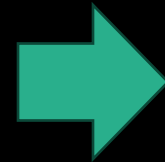
```
00111100
01100110
11000011
11000011
11111111
11000011
11000011
11000011
11000011
```



Interpretando:

0 = ■

1 = □



2 - Começando com uma imagem extremamente simples FIAP

Por exemplo, podemos representar a letra **A**:

00111100
01100110
11000011
11000011
11111111
11000011
11000011
11000011



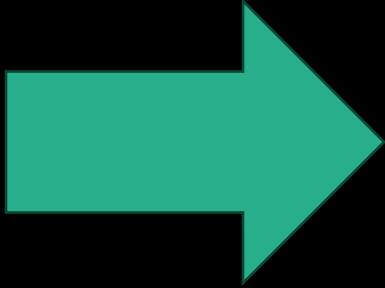
Aqui está a ideia mais importante da aula:

Uma imagem pode ser armazenada como uma sequência de números.

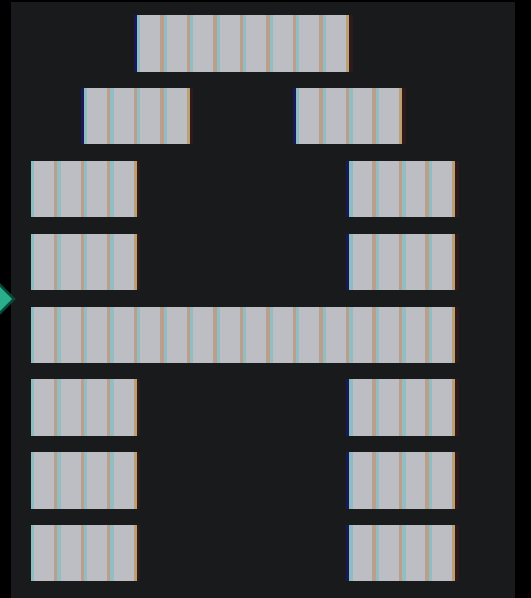
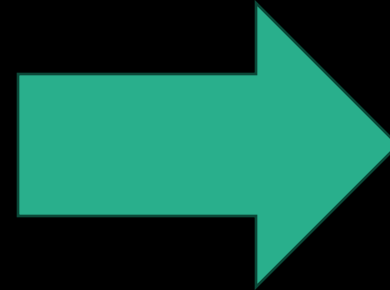
Os números podem ser representados em binário.

3 - Primeiro experimento em Python

```
imagem = [  
[0,0,1,1,1,1,0,0],  
[0,1,1,0,0,1,1,0],  
[1,1,0,0,0,0,1,1],  
[1,1,0,0,0,0,1,1],  
[1,1,1,1,1,1,1,1],  
[1,1,0,0,0,0,1,1],  
[1,1,0,0,0,0,1,1],  
[1,1,0,0,0,0,1,1],  
[1,1,0,0,0,0,1,1]  
]
```



```
[0, 0, 1, 1, 1, 1, 0, 0]  
[0, 1, 1, 0, 0, 1, 1, 0]  
[1, 1, 0, 0, 0, 0, 1, 1]  
[1, 1, 0, 0, 0, 0, 1, 1]  
[1, 1, 0, 0, 0, 0, 1, 1]  
[1, 1, 1, 1, 1, 1, 1, 1]  
[1, 1, 0, 0, 0, 0, 1, 1]  
[1, 1, 0, 0, 0, 0, 1, 1]  
[1, 1, 0, 0, 0, 0, 1, 1]
```



```
for linha in imagem:  
    print(linha)
```

```
for linha in imagem:  
    for pixel in linha:  
        if pixel == 1:  
            print("█", end="")  
        else:  
            print(" ", end="")  
    print()
```

4 - Transformando a imagem em uma sequência binária

Podemos transformar cada linha:

0 0 1 1 1 1 0 0

em:

00111100

Essa sequência possui:
 $8 \times 8 = 64$ bits

Ou seja:

Nossa imagem de 8×8 pixels pode ser armazenada utilizando apenas 64 bits.

Em Python:

```
<> Python

for linha in imagem:
    binario = ""

    for pixel in linha:
        binario += str(pixel)

    print(binario)
```

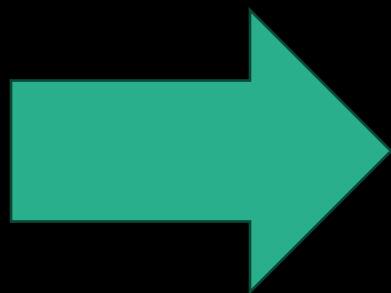
Resultado:

0011110001100110110000111100001111111111110000111100001111000011

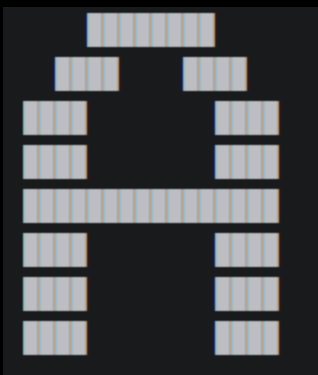
00111100
01100110
11000011
11000011
11111111
11000011
11000011
11000011

5 - Agora vamos fazer o caminho inverso

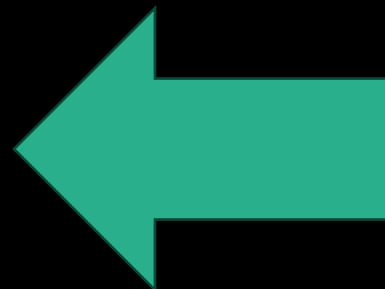
00111100
01100110
11000011
11000011
11111111
11000011
11000011
11000011
11000011



O computador precisa interpretar esses bits.
Vamos fazer isso em Python.



```
dados = [  
    0b00111100,  
    0b01100110,  
    0b11000011,  
    0b11000011,  
    0b11111111,  
    0b11000011,  
    0b11000011,  
    0b11000011  
]  
  
for byte in dados:  
    binario = format(byte,  
        "08b")  
    for bit in binario:  
        if bit == "1":  
            print("█", end="")  
        else:  
            print(" ", end="")  
  
print()
```



6 - Converter Imagem PNG em Binário

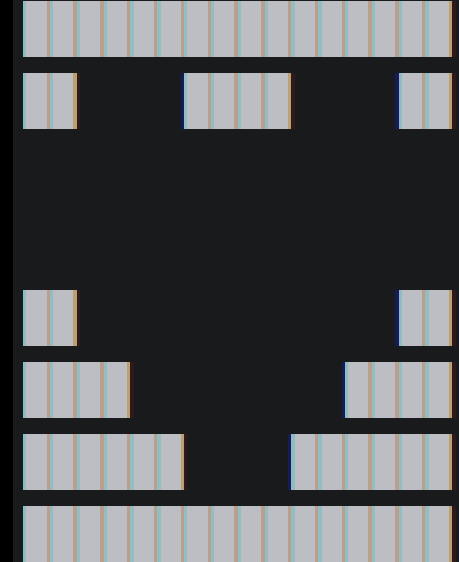
FIAP



Instale primeiro:

```
pip install pillow
```

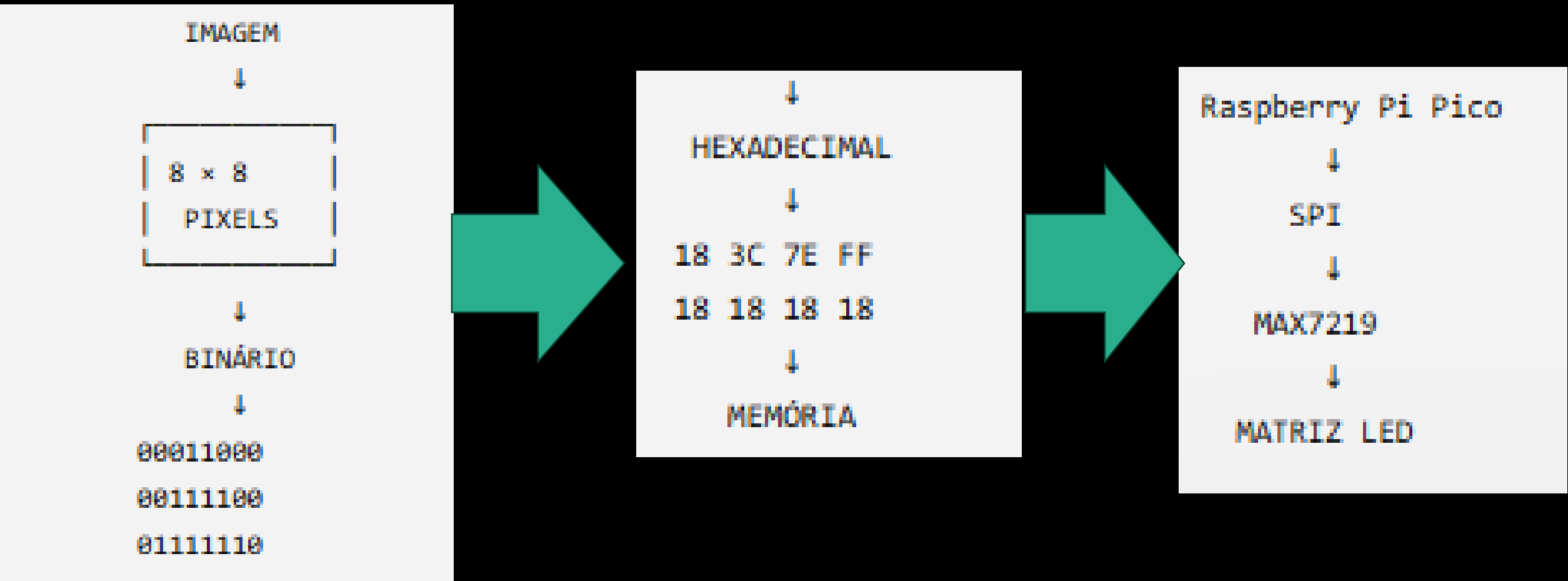
MATRIZ 8 x 8



BINÁRIO	HEXADECIMAL
-----	-----
11111111	0xFF
10011001	0x99
00000000	0x00
00000000	0x00
10000001	0x81
11000011	0xC3
11100111	0xE7
11111111	0xFF

Depois coloque uma imagem chamada `imagem.png` na mesma pasta do programa.

7 - Representações



8 - Agora vamos para o Raspberry Pi Pico + Wokwi

No Wokwi, vamos utilizar:
Raspberry Pi Pico
+
MAX7219 8x8 LED Matrix

Ligações

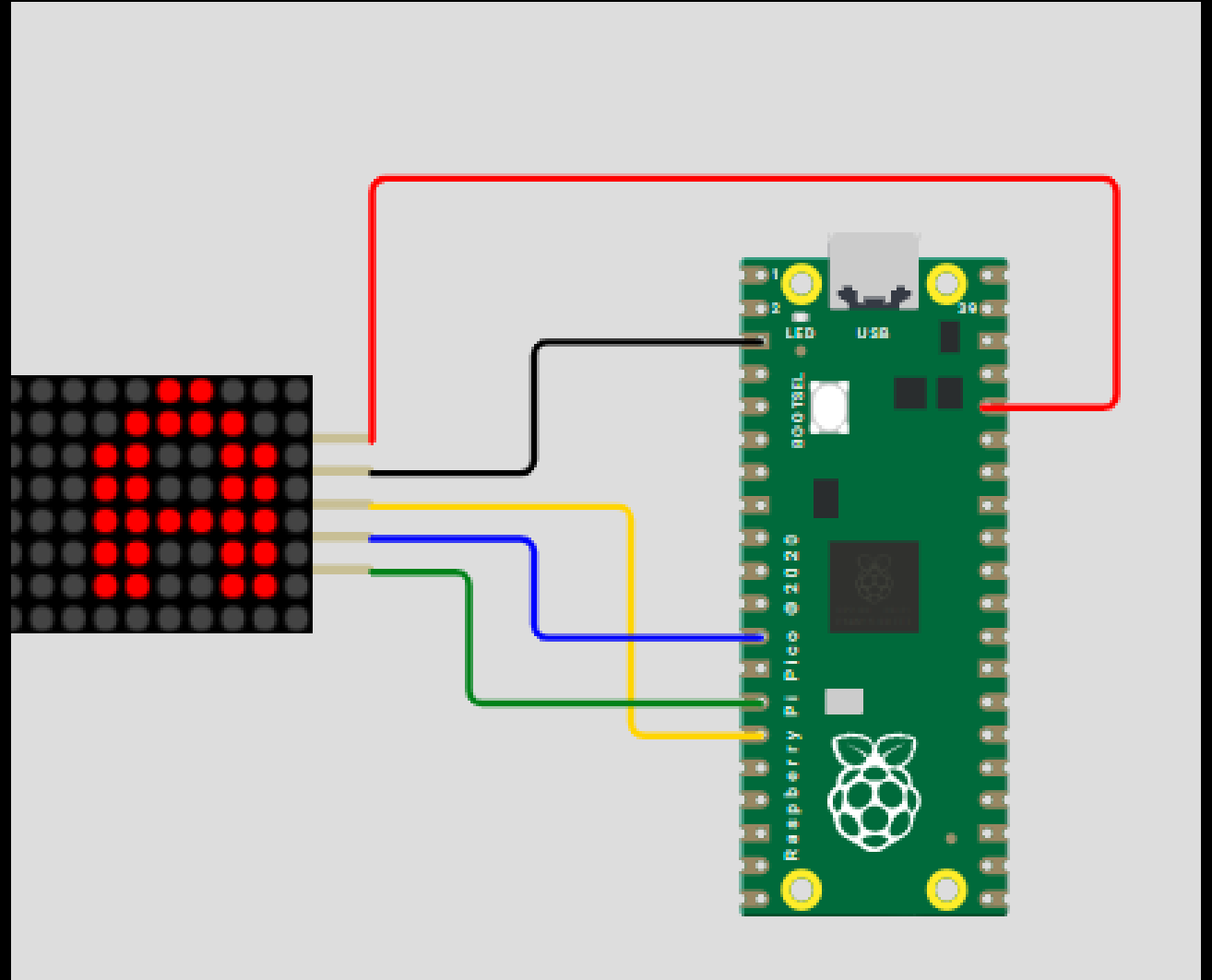
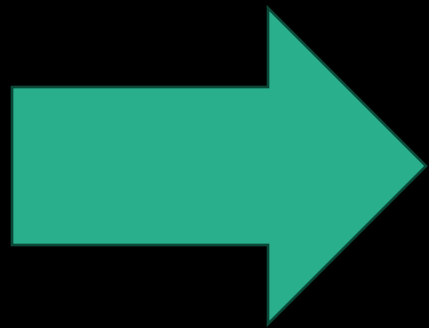
Matrix de Led possui os pinos:

	MAX7219	Raspberry Pi Pico
VCC	VCC	3V3
GND	GND	GND
DIN	DIN	GP11
CS	CLK	GP10
CLK	CS	GP9

8 - Agora vamos para o Raspberry Pi Pico + Wokwi

Podemos enviar exatamente os 8 bytes que acabamos de produzir no Python.

```
imagem = [  
    0x18,  
    0x3C,  
    0x66,  
    0x66,  
    0x7E,  
    0x66,  
    0x66,  
    0x00  
]
```



8 - Resultado final Raspberry Pi Pico + Wokwi FIAP

O Python produziu.

0x18
0x3C
0x66
0x66
0x7E
0x66
0x66
0x00

O Pico recebe esses valores como dados.

Por exemplo:

0x18

em binário:

00011000

O MAX7219 interpreta esses 8 bits como uma linha da matriz.

vira:



Então:

0x18



00011000



Depois:

0x3C



00111100



8 - Agora e com você

Atividade

Cada um deverá escolher uma imagem simples em PNG, por exemplo:

- Quadrado
- Círculo
- Bola
- ★ Estrela
- ♥ Coração
- ↑ Seta
- Letra do Nome

A imagem deve ser:

- simples;
- preto e branco;
- sem detalhes pequenos;
- preferencialmente com fundo branco;
- reconhecível mesmo com apenas 8×8 pixels.

8 - Agora e com você

Objetivo

O aluno deverá pegar uma imagem simples em preto e branco, reduzi-la para uma matriz **8×8**, converter seus pixels em:

imagem → binário → hexadecimal → bytes

e depois utilizar os 8 bytes no Raspberry Pi Pico para reconstruir a imagem em uma matriz de LEDs.